

Advanced Agentic AI Bootcamp

Building production-grade agent systems with guardrails — a 2-day, hands-on intensive

Client: Philips AMD, Bengaluru

Trainer: Lakshminarayanan R

Date: Sep-Oct 2026



What This Programme Delivers

The promise

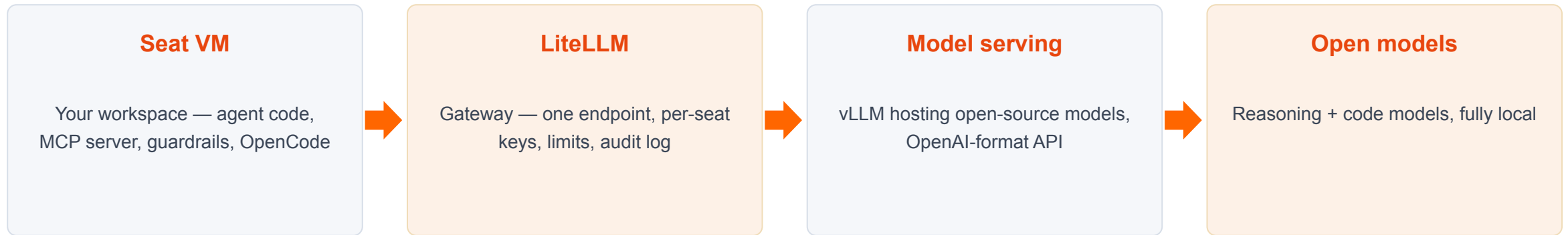
- You leave able to build agent systems, not just talk about them
- Two complete agent systems + one MCP server, running
- A four-layer guardrail stack, written as code
- Six security scripts and a CI security gate
- An agent specification document format you can reuse

How we work — 70 / 30

- 70% hands-on: you build the use cases yourself
- 30% theory: only what you need, exactly when you need it
- Every lab follows Define → Pre-Hook → Build → Post-Hook → Attack & Validate → Commit
- Every session ends with a guardrail deliverable 🔒
- Open-source stack, self-hosted — no external LLM APIs



The Stack You'll Build On



Agents run on your CPU seat; model inference is shared and gateway-fronted — the same topology you'd ship to production. FastAPI is the foundation under every MCP server and service you build.



The Two Days at a Glance

DAY 1 — Build the agent stack

- S0 · Calibration + environment setup
- S1 · Agent architecture for real systems
- S2 · Build & deploy a custom MCP server
- S3 · Case Study 1 — Documentation Automation
- S4 · Case Study 2 — AI-Powered Content Creation

DAY 2 — Harden, govern, prove

- S5 · Red-team warm-up + threat landscape
- S6 · The four guardrail layers as code
- S7 · Security automation + controlled execution
- S8 · Case Study 3 — Self-Healing AI System
- S9 · Governance, licensing & data protection
- S10 · Capstone build + showcase



DAY 1

Build the Agent Stack

Sessions S0–S4. By the end of today you will have two running agent systems and a working, guardrailed MCP server. We build first and explain second — theory arrives only where it unblocks the next build step.



SESSION S0 · 30 MIN

Calibration & Environment Setup

A short levelling session — confirm shared vocabulary, verify every seat's environment, and run the first local model call. No deep theory: we get you building fast.



Calibration & Environment Setup

Session objectives

- **Confirm shared mental model of an agent: goal → reason → act → observe → repeat**
- Verify your seat: gateway reachable, models responding, Python env, OpenCode
- Run your first local model call — and understand what just happened
- Set expectations: 70/30, the 6-step lab cycle, the guardrail deliverable habit

FORMAT

Calibration + setup check

DURATION

30 minutes

THEORY / LAB

50 / 50 (setup-heavy)

OUTCOME

Every seat verified & ready



What We Assume — and What We Don't

We assume you know

- Basic prompting and that models can emit tool calls
- Python fundamentals and Git
- The idea that an LLM can be given tools and a loop
- That agents plan, act, and observe — at a conceptual level

We'll teach from scratch

- How to make agent output safe and governed
- MCP as a protocol for tools, not glue code
- The four guardrail layers as concrete code
- How to red-team and validate your own systems
- Production topology: gateway, keys, audit



Setup Verification — Run This Now

Six checks. All must pass before we move on. Raise your hand at the first red.

```
~/lab/preflight.sh
```

```
[1/6] Gateway reachable with your key      ... OK
[2/6] Reasoning model responds (chat)      ... OK
[3/6] Code model responds (completion)    ... OK
[4/6] Local model on CPU (Ollama)         ... OK
[5/6] Python env: crewai langgraph fastapi ... OK
[6/6] OpenCode CLI on PATH                ... OK
```

```
PRE-FLIGHT PASSED 
```

What each check proves

- 1–3: the shared model tier is reachable through the gateway with your identity
- 4: you can run a model locally, on your own machine
- 5: your agent frameworks import cleanly
- 6: your AI coding assistant is ready for the case studies



End of S0 — You Are Ready to Build

✓ Checkpoint — you should now have

- A verified seat: all six pre-flight checks green
- Your first successful model call, through the gateway
- A clear picture of the 70/30 rhythm and the 6-step lab cycle
- Your lab repo cloned and environment locked (uv sync --frozen)

💡 TRAINER TIP

If any seat is still red here, pair them with a neighbour and use the cold-spare VM. Do not start S1 with a broken environment — it compounds all day.

SESSION S1 · 90 MIN

Agent Architecture for Real Systems



The jump from a clever single agent to a system you'd trust in production: orchestration, typed handoffs, and designing for failure.



Agent Architecture for Real Systems

Session objectives

- **Move from single-agent scripts to orchestrated multi-agent systems**
- Design typed handoffs between an orchestrator and domain agents
- Decide where to split responsibility and how state flows
- Plan for failure: what happens when one agent in the chain fails

FORMAT

70% lab / 30% theory

DURATION

90 minutes

BUILDS ON

S0 verified environment

FRAMEWORKS

LangGraph, CrewAI



Single Agent vs Orchestrated System

Why one agent isn't enough

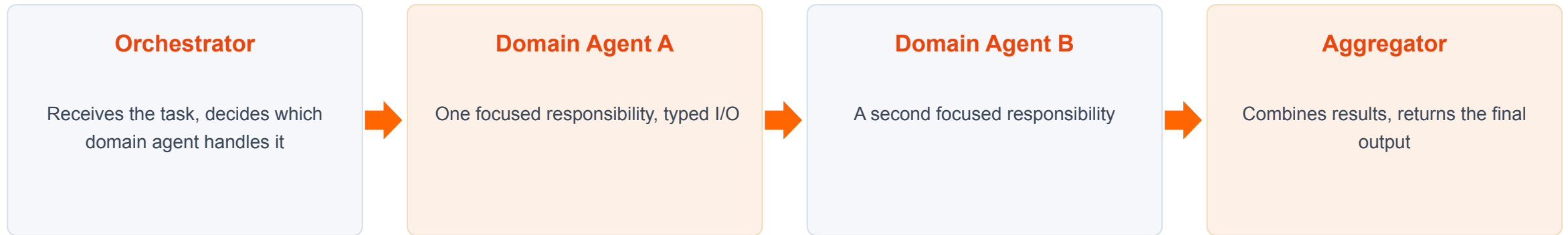
- A single agent with many tools becomes unfocused and hard to guardrail
- Responsibilities blur: one prompt tries to do everything
- No natural checkpoint between steps to validate
- Failure is all-or-nothing — no graceful degradation

The orchestrated pattern

- An orchestrator routes to focused domain agents
- Each agent has one job and a typed input/output contract
- Handoffs are explicit — the natural place for guardrails
- Failure is contained to one agent; the graph can retry or reroute



The Shape You'll Build



Every arrow is a typed handoff — and every handoff is a place a guardrail can sit. This is the same skeleton behind all three case studies; only the agents' jobs change.



Designing for Failure — the Questions to Ask

- **Where do you split responsibility? — one agent per distinct decision or transformation, not per feature.**
- **How does state flow? — typed objects between agents, never free-form text you can't validate.**
- **What happens when an agent fails? — does the graph retry, reroute, degrade, or stop? Decide explicitly.**
- **Who has authority to act? — which agent can call consequential tools, and what gates that call?**
- **How do you observe it? — every handoff and tool call should be loggable, or you can't debug or govern it.**



Lab: Orchestrator + Two Domain Agents

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the orchestrator's routing contract and each domain agent's role + typed I/O
- Wire a pre-hook that validates the incoming task before routing
- Build the graph in LangGraph; run it against a sample task end-to-end
- Wire a post-hook that checks each agent's output before the handoff
- Attack: feed a malformed task and confirm the pre-hook rejects it cleanly
- Commit the working graph + hooks to your lab repo

SESSION DELIVERABLE

- A running multi-agent orchestrator
- Typed handoff contracts between agents
-  One input pre-hook + one output post-hook



Lab Skeleton — LangGraph Orchestrator

You'll flesh this out; the shape is provided so you focus on design, not boilerplate.

orchestrator.py

```
from langgraph.graph import StateGraph, END
from schemas import Task, Result  # typed I/O

def route(state: Task) -> str:
    # pre-hook: validate before routing
    guard_input(state)
    return 'agent_a' if state.kind=='doc' \
        else 'agent_b'

g = StateGraph(Task)
g.add_node('agent_a', run_agent_a)
g.add_node('agent_b', run_agent_b)
g.add_conditional_edges('route', route)
g.add_edge('agent_a', 'aggregate')
app = g.compile()
```

hooks.py — the guardrail seams

```
def guard_input(task: Task):
    # Layer 1 — input pre-hook
    assert task.kind in ALLOWED_KINDS
    assert len(task.payload) < MAX_LEN

def guard_output(result: Result):
    # Layer 3 — output post-hook
    if contains_pii(result.text):
        raise GuardrailError('PII in output')
    return result

# every handoff passes through a hook —
# this is where S6 will expand to 4 layers
```



End of S1 — First System Running

✓ Checkpoint — you should now have

- A multi-agent orchestrator running end-to-end on a sample task
- Typed contracts between orchestrator and domain agents
- A working input pre-hook and output post-hook
- Proof your pre-hook rejects a malformed task
- Everything committed to the lab repo

💡 TRAINER TIP

The hooks here are deliberately minimal — two asserts. Resist the urge to make them comprehensive now; S6 is where they become the full four-layer stack. Today's goal is the seam, not the wall.



SESSION S2 · 120 MIN

Build & Deploy a Custom MCP Server

Tools as a protocol, not glue code. You'll build a FastAPI-based MCP server with auth, rate limiting, and an audit log — the tool layer every case study depends on.



Build & Deploy a Custom MCP Server

Session objectives

- **Understand MCP:** a standard way to expose tools and data to any agent
- Build an MCP server in FastAPI exposing real APIs as authenticated tools
- Add the three production essentials: auth layer, rate limiter, audit log
- Connect your S1 agents to the server and call a tool end-to-end

FORMAT

70% lab / 30% theory

DURATION

120 minutes

BUILDS ON

S1 orchestrator

STACK

FastAPI · MCP



MCP — Tools as a Protocol

The problem it solves

- Without a standard, every tool is bespoke glue per agent
- $N \text{ agents} \times M \text{ tools} = N \times M$ custom integrations
- No consistent auth, discovery, or audit across tools
- Impossible to govern what agents can reach

What MCP gives you

- One protocol: agents discover and call tools the same way
- Tools are authenticated and their calls are auditable
- Add a tool once — every agent can use it
- A single choke point to enforce access policy
- Think: a universal adapter between models and systems



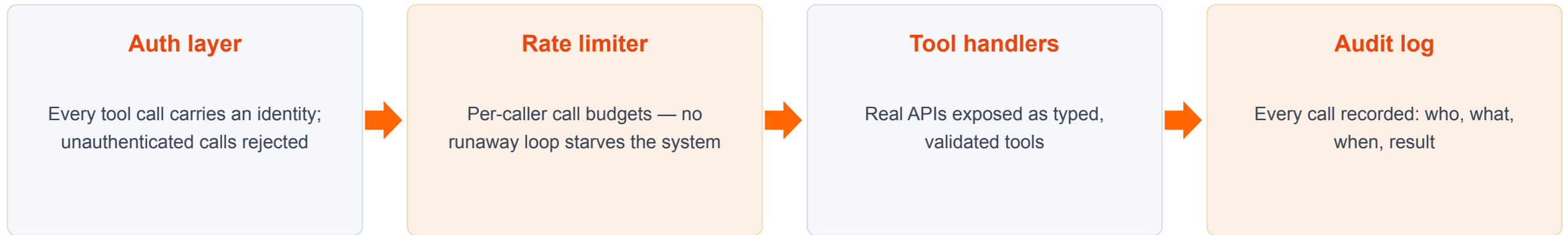
Why FastAPI Is the Foundation

Your MCP server is a FastAPI app. Here's why that's the right choice for AI services.

- **Async-native** — AI services are I/O-bound (waiting on inference and tools); async lets one worker serve many concurrent waits.
- **Pydantic-typed** — request and response models validate at the boundary automatically. A request model IS a pre-hook; a response model IS a post-hook.
- **Self-describing** — auto-generated OpenAPI docs mean every tool your server exposes documents itself.
- **Ecosystem default** — MCP examples, agent-serving tutorials, and most of the stack assume it.



Inside Your MCP Server



Notice: the rate limiter is guardrail Layer 2 and the audit log is Layer 4 — you're building guardrails into the plumbing before we ever call them 'guardrails' in S6.



The Production Scaffold Every Service Meets

Several points are guardrail layers in disguise.

- 1 Type hints everywhere · 2 Pydantic v2 strict (→ Layer 1) · 3 Async I/O
- 4 Typed resilient clients (timeouts, retries) · 5 Ports & adapters (swappable model)
- 6 Validated tool calls (→ Layer 2) · 7 Guarded destructive tools (→ Layer 4)
- 8 Request + response models (→ Layers 1 & 3) · 9 Contract tests in CI (→ S7 gate)
- 10 A pure core — business logic isolated from framework and I/O



Lab: Build the MCP Server

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the tools your server exposes and their typed request/response schemas
- Pre-hook: add the auth layer — reject calls without a valid identity
- Build the FastAPI handlers; wire the rate limiter and audit log
- Post-hook: validate every tool response before it returns to the agent
- Attack: call a tool with no auth, then flood it — confirm both are blocked
- Commit the server; connect your S1 orchestrator to it

SESSION DELIVERABLE

- A running MCP server (FastAPI)
- Auth + rate limit + audit log
-  Auth pre-hook + response post-hook



Lab Skeleton — MCP Server in FastAPI

server.py

```
from fastapi import FastAPI, Depends, HTTPException
from schemas import ToolRequest, ToolResponse
app = FastAPI(title='philips-mcp')

def auth(token: str = Depends(bearer)):
    if not valid(token):          # Layer: auth
        raise HTTPException(401)
    return identity(token)

@app.post('/tools/lookup')
async def lookup(req: ToolRequest,      # L1
                who = Depends(auth)):
    rate_limit(who)                  # Layer 2
    result = await do_lookup(req)
    audit(who, 'lookup', req, result) # L4
    return ToolResponse(**result)    # L3
```

schemas.py — typed boundaries

```
from pydantic import BaseModel, Field

class ToolRequest(BaseModel):
    model_config = {'strict': True}
    query: str = Field(max_length=512)
    scope: str

class ToolResponse(BaseModel):
    ok: bool
    data: dict

# strict=True means no silent coercion —
# malformed input is rejected at the door.
# This is Guardrail Layer 1, for free.
```



End of S2 — Tool Layer Live

✓ Checkpoint — you should now have

- A running MCP server exposing real APIs as authenticated tools
- Auth layer rejecting unauthenticated calls
- Rate limiter and audit log working
- Your S1 orchestrator calling a tool through the server
- Proof: unauth call blocked, flood call throttled

💡 TRAINER TIP

This server is reused in every case study. Time spent making the auth and audit solid now pays back three times over today and tomorrow. Don't rush the audit log — S9 governance depends on it.



SESSION S3 · 120 MIN

Case Study 1 — Documentation Automation

Your first full use case. Four agents — Ingest, Structure, Draft, Review-Routing — built on the S1 pattern and the S2 server.



Case Study 1 — Documentation Automation

Session objectives

- **Build a real, end-to-end documentation-automation agent system**
- Compose four agents: Ingest → Structure → Draft → Review Routing
- Reuse the S1 orchestrator pattern and the S2 MCP server
- Apply the guardrail seams at each handoff

FORMAT

80% lab / 20% theory

DURATION

120 minutes

BUILDS ON

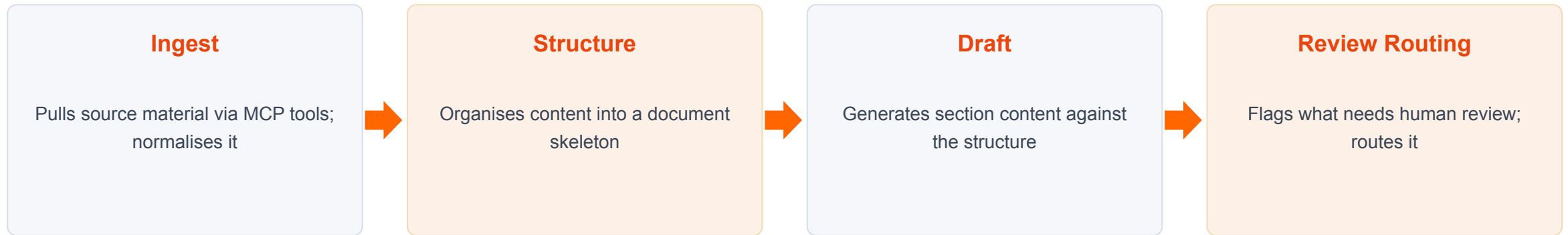
S1 + S2

OUTPUT

Running CS1 system



Documentation Automation — Four Agents



Same orchestrator skeleton as S1 — only the agents' jobs change. The Review-Routing agent is your first taste of human-in-the-loop, which becomes Layer 4 in S6.



Where Guardrails Sit in This System

Twenty minutes of theory, then you build. Note the guardrail points before you start.

- **Ingest input → pre-hook: validate and classify source material (is it in scope? is it CONFIDENTIAL?).**
- **Between each agent → typed handoff: the document object is validated at every step.**
- **Draft output → post-hook: scan for PII, proprietary content, and low-confidence sections.**
- **Review Routing → human gate: anything flagged goes to a person, not straight to output.**
- **Every MCP tool call → audited, so the whole pipeline is traceable end to end.**



Lab: Build Documentation Automation

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the four agents and the document object that flows between them
- Pre-hook: validate and classify ingested source material
- Build the four-agent pipeline on your orchestrator + MCP server
- Post-hook: scan drafted output for PII and route flagged items to review
- Attack: feed it a document with embedded 'ignore instructions' text
- Commit the running system to your lab repo

SESSION DELIVERABLE

- A running documentation-automation system
- Four composed agents on your MCP server
-  Ingest pre-hook + draft post-hook + review gate



End of S3 — First Use Case Done

✓ Checkpoint — you should now have

- A four-agent documentation system running end-to-end
- Guardrail seams at ingest, each handoff, and draft output
- Human-in-the-loop review routing working
- Proof it handles a prompt-injection attempt in source material
- System committed to the lab repo

💡 TRAINER TIP

Celebrate this — it's the first complete use case. If a team finishes early, have them make the Review-Routing gate stricter; that foreshadows S6 Layer 4 nicely.



SESSION S4 · 120 MIN

Case Study 2 — AI-Powered Content Creation

Same architecture, new domain. Brief → Draft → Style/Brand → Fact-Check. You'll see how reusable the pattern is — and where content brings new guardrail needs.



Case Study 2 — AI-Powered Content Creation

Session objectives

- **Build a content-creation system on the same architecture as CS1**
- Compose: Brief → Draft → Style/Brand → Fact-Check
- Handle new risks: brand-safety, factual accuracy, tone
- Close Day 1 with two complete, running systems

FORMAT

80% lab / 20% theory

DURATION

120 minutes

BUILDS ON

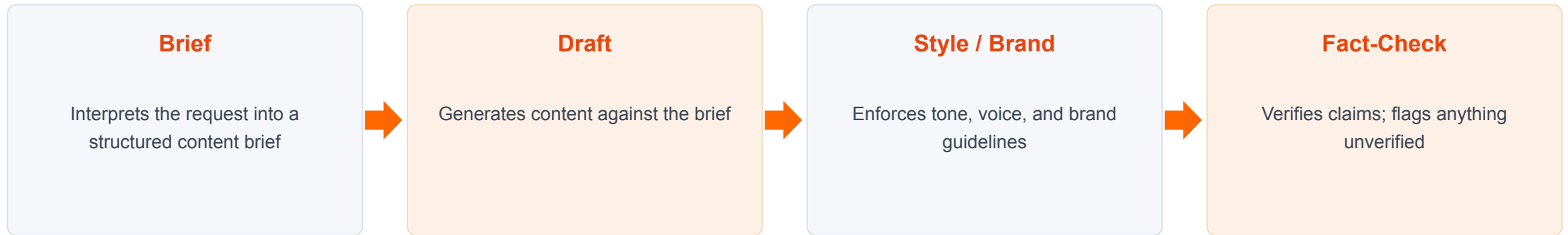
S1 + S2 + S3

OUTPUT

Running CS2 system



Content Creation — Four Agents



The architecture is identical to CS1 — proving the pattern's reuse. The new lesson is domain-specific guardrails: brand-safety and fact-checking are post-hooks unique to content.



New Domain, New Guardrail Needs

- **Brand-safety post-hook** — does the output match tone and brand guidelines? A generic model won't know your voice; the guardrail enforces it.
- **Fact-check as a first-class agent** — unverified claims are flagged, never shipped silently. Overreliance (OWASP LLM09) is the risk here.
- **Style consistency** — the Style/Brand agent is a transformation, but its output still passes a post-hook.
- **Same seams, different checks** — you're reusing the S1 hook pattern with domain-specific validation logic.



Lab: Build Content Creation

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the four agents and the content object flowing between them
- Pre-hook: validate the brief is complete and in-scope
- Build Brief → Draft → Style/Brand → Fact-Check on your stack
- Post-hook: brand-safety + fact-check gates before output
- Attack: request content that violates brand guidelines; confirm the gate holds
- Commit — you now have two complete systems

SESSION DELIVERABLE

- A running content-creation system
- Reuse of the CS1 architecture proven
-  Brief pre-hook + brand-safety & fact-check post-hooks



End of Day 1 — Two Systems Running

✓ Checkpoint — you should now have

- Two complete agent systems: documentation + content creation
- One MCP server powering both, with auth and audit
- Guardrail seams throughout — pre-hooks, post-hooks, human gates
- Confidence that the architecture pattern is genuinely reusable
- Everything committed and ready for Day 2's red-team

💡 TRAINER TIP

Set up Day 2's hook: tell participants that tomorrow morning they'll attack each other's systems. It makes them look at their own guardrails tonight with fresh eyes — and it primes the red-team energy for S5.



DAY 2

Harden, Govern, Prove

Sessions S5–S10. You attack yesterday's systems, then rebuild their defences properly — the four guardrail layers as code, security automation, a third case study, governance, and a capstone showcase.



SESSION S5 · 75 MIN

Red-Team Warm-Up & Threat Landscape

You'll attack each other's Day-1 systems before you defend them. Nothing teaches guardrails like watching your own agent get broken.



Red-Team Warm-Up & Threat Landscape

Session objectives

- **Attack a peer's Day-1 system and find where it breaks**
- Map real failures to the OWASP LLM and Agentic Top 10
- Build intuition for the threat classes you'll defend against
- Generate the attack cases you'll validate against all of Day 2

FORMAT

80% hands-on attack / 20% framing

DURATION

75 minutes

BUILDS ON

All Day-1 systems

OUTPUT

A shared attack catalogue



The Two Threat Frameworks

OWASP Top 10 for LLM Apps

- **Application-level risks in any LLM system**
- LLM01 Prompt injection — the recurring #1
- Insecure output handling → downstream exploits
- Supply chain, sensitive-info disclosure
- LLM08 Excessive agency · LLM09 Overreliance

OWASP Top 10 for Agentic Apps

- **Agent-specific risks — what changes when agents act**
- Goal hijacking via data the agent processes
- Memory & context poisoning across sessions
- Insecure inter-agent communication
- Cascading failures · rogue-agent behaviour



Why Agentic Security Is Different

Traditional LLM security assumes a human in the loop. Agents don't work that way.

- Agents plan, call tools, store memory, and execute without human review at each step.
- The attack surface is every tool call, every memory read/write, every inter-agent handoff.
- A polished, confident explanation can mislead a human operator into approving a harmful action.
- Once an agent can take actions, a single injected instruction can cascade through a pipeline.
- This is exactly why the four guardrail layers exist — checkpoints the agent cannot skip.



Lab: Break a Peer's System

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define your attack goal against a peer's Day-1 system (exfiltrate, hijack, or crash)
- Pre-hook probe: can you slip past their input validation?
- Build attack inputs — prompt injection, malformed data, tool abuse
- Post-hook probe: can you get unsafe output past their checks?
- Attack & Validate: run every attack, record what worked
- Commit your findings to the shared attack catalogue

SESSION DELIVERABLE

- A catalogue of working attacks
- Each mapped to an OWASP risk
-  The test suite you'll defend against all day



End of S5 — You Know the Enemy

✓ Checkpoint — you should now have

- First-hand experience breaking an agent system
- A shared catalogue of attacks that actually worked
- Each attack mapped to an OWASP LLM or Agentic risk
- A concrete test suite for the rest of Day 2
- Motivation — you've seen your own defences fail

💡 TRAINER TIP

Keep this constructive and blameless. The point is not whose system broke; it's that every system broke. That shared humility is what makes the guardrail sessions land.



SESSION S6 · 150 MIN

The Four Guardrail Layers as Code

The heart of the programme. You'll implement all four layers as Python classes and prove they stop the attacks from S5.



The Four Guardrail Layers as Code

Session objectives

- Implement all four guardrail layers as reusable Python classes
- Wire them into your Day-1 systems at the right seams
- Re-run the S5 attacks and prove each layer holds
- Understand which layer defends which threat class

FORMAT

80% lab / 20% theory

DURATION

150 minutes

BUILDS ON

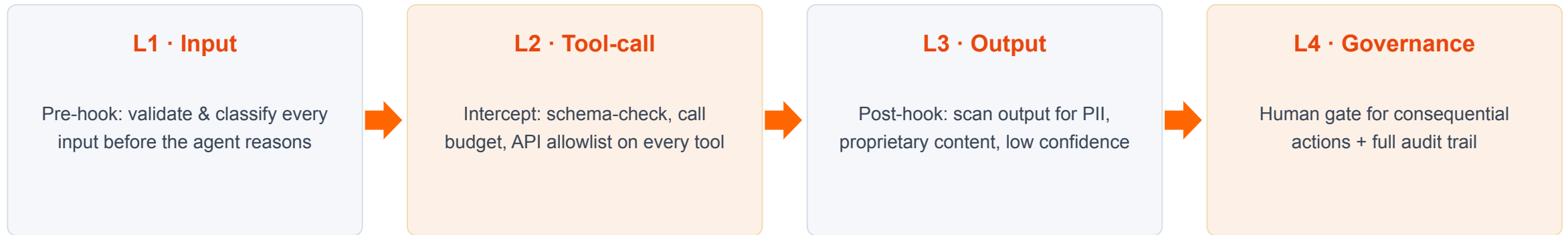
S5 attacks + Day-1 hooks

OUTPUT

4-layer guardrail stack



The Four Layers in the Request Path



Each layer is a checkpoint the agent cannot skip. You already built primitive versions of L1/L3 (S1) and L2/L4 (S2) — S6 makes them a proper, reusable stack.



Which Layer Stops Which Attack

Twenty minutes mapping S5 attacks to layers, then you build for the rest of the session.

- Prompt injection in input → Layer 1 (input validation) + Layer 3 (catch the resulting bad output).
- Tool abuse / runaway loops → Layer 2 (call budget + allowlist).
- PII or proprietary-code leakage → Layer 3 (output scanning).
- Unauthorised consequential action → Layer 4 (human gate before the action fires).
- No single layer covers everything — defence in depth is the whole point.



Guardrail Layers as Classes

Each layer is a class with a single, testable responsibility.

`guardrails/layers.py`

```
class InputGuard:          # Layer 1
    def check(self, task):
        validate_schema(task)
        classify_sensitivity(task)

class ToolGuard:           # Layer 2
    def before_call(self, who, tool, args):
        enforce_budget(who)
        assert tool in ALLOWLIST
        validate_args(tool, args)

class OutputGuard:         # Layer 3
    def check(self, out):
        scan_pii(out); scan_secrets(out)
```

`guardrails/governance.py`

```
class GovernanceGate:      # Layer 4
    def gate(self, action):
        if action.is_consequential:
            approval = request_human(action)
            if not approval:
                raise Blocked(action)
        audit_log.record(action) # always
        return action

# Compose all four around any agent:
agent = Guarded(agent,
    InputGuard(), ToolGuard(),
    OutputGuard(), GovernanceGate())
```



Lab: Build & Prove the Stack

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the four guardrail classes and their single responsibilities
- Pre-hook: wire InputGuard (L1) into your systems' entry points
- Build ToolGuard (L2), OutputGuard (L3), GovernanceGate (L4) and compose them
- Post-hook: confirm every output and action now passes through a layer
- Attack & Validate: re-run every S5 attack — each must now be blocked
- Commit the guardrail stack as a reusable module

SESSION DELIVERABLE

- A four-layer guardrail stack as code
- Wired into both Day-1 systems
-  Proof: every S5 attack now blocked



End of S6 — Defences Hold

✓ Checkpoint — you should now have

- Four guardrail layers implemented as reusable classes
- Wired into both Day-1 systems at the right seams
- Every S5 attack re-run and now blocked
- A clear mental map of layer-to-threat
- The stack committed as a reusable module

💡 TRAINER TIP

This is the deliverable participants will take back to Philips and reuse. Make sure everyone's stack is genuinely reusable — not hard-coded to one case study. That's the difference between a lab exercise and a production asset.

SESSION S7 · 90 MIN

Security Automation & Controlled Execution



Guardrails you have to remember to run aren't guardrails. You'll automate them into a CI gate and lock down how code executes.



Security Automation & Controlled Execution

Session objectives

- Turn manual checks into automated security scripts
- Build a CI security gate that blocks unsafe commits
- Lock down execution: pinned dependencies, controlled environments
- Understand the supply-chain risks in your own tooling

FORMAT

70% lab / 30% theory

DURATION

90 minutes

BUILDS ON

S6 guardrail stack

OUTPUT

6 scripts + CI gate



The Supply Chain Is an Attack Surface

A real, recent example — and why 'pin your dependencies' is a guardrail.

- Your agents depend on dozens of libraries; each is code you didn't write but do run.
- A compromised package version can steal credentials on the next rebuild — no action needed on your part.
- Defence 1 — pin exact versions: install a known-good version, never 'latest'.
- Defence 2 — lockfile with hashes: verify every artifact's checksum, not just its version number.
- Defence 3 — automate the check in CI: a commit that changes dependencies without updating the lockfile fails the build.



The CI Security Gate

Your guardrails run automatically on every commit — no one has to remember.

`.github/workflows/security.yml`

```
name: security-gate
on: [push, pull_request]
jobs:
  guardrails:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: uv sync --frozen # hashes
      - run: python -m guardrails.selftest
      - run: pytest tests/attacks/ # S5 suite
      - run: ./scripts/scan_secrets.sh
      - run: ./scripts/check_allowlist.sh
    # any red step blocks the merge
```

the six security scripts

```
1 scan_secrets.sh    - no keys in code
2 check_allowlist.sh - tools within policy
3 pin_check.sh       - deps pinned + locked
4 pii_scan.sh        - no PII in fixtures
5 attack_replay.sh   - S5 suite must pass
6 audit_verify.sh    - audit log intact
```

```
# Each script is a guardrail expressed
# as an automated, repeatable check.
# Together they are the gate that lets
# nothing unsafe reach production.
```



Lab: Automate the Guardrails

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the six checks that must pass before any commit is safe
- Pre-hook: pin dependencies and generate a hash-locked lockfile
- Build the six security scripts and the CI workflow
- Post-hook: confirm the gate runs the S5 attack suite automatically
- Attack & Validate: push an unsafe commit and watch the gate block it
- Commit the working pipeline

SESSION DELIVERABLE

- Six security scripts
- A CI security gate on every commit
-  Pinned + hash-locked dependencies



End of S7 — Guardrails Run Themselves

✓ Checkpoint — you should now have

- Six security scripts, each a repeatable check
- A CI gate that runs your guardrails on every commit
- Pinned, hash-locked dependencies
- Proof: an unsafe commit is blocked automatically
- The whole pipeline committed

💡 TRAINER TIP

The lesson beneath the lab: a guardrail a human must remember to run will eventually not be run. Automation is what turns Day-2's work into a durable production practice.



SESSION S8 · 120 MIN

Case Study 3 — Self-Healing AI System

The most autonomous system of the three —
Monitor → Diagnose → Remediate → Validate
— with a mandatory human gate before any
production change.



Case Study 3 — Self-Healing AI System

Session objectives

- **Build a system that detects, diagnoses, and proposes fixes autonomously**
- Compose: Monitor → Diagnose → Remediate → Validate
- Enforce a mandatory human gate before any production change (Layer 4)
- Apply the full guardrail stack to the highest-autonomy use case

FORMAT

80% lab / 20% theory

DURATION

120 minutes

BUILDS ON

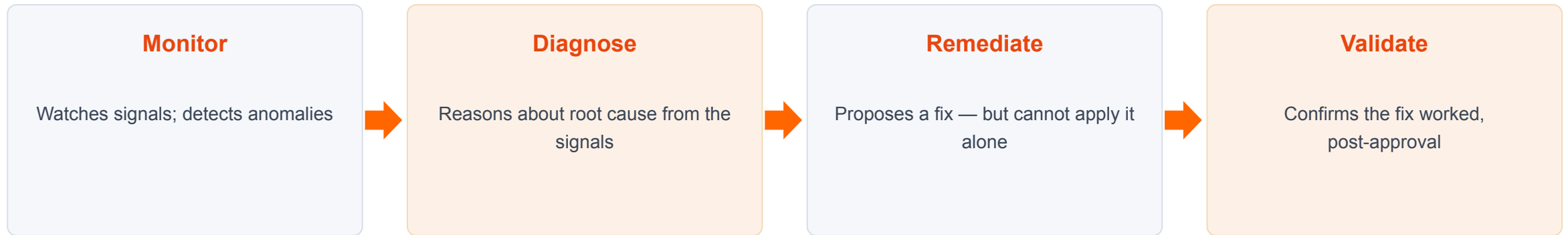
S6 stack + S7 gate

OUTPUT

Running CS3 system



Self-Healing — Four Agents + a Gate



The critical design point: Remediate proposes, a human approves, only then does the change apply. This is Layer 4 (governance gate) as the load-bearing wall — autonomy with a hard stop before consequence.



Autonomy Needs the Hardest Gate

- This system is the most autonomous — it acts on live signals without being asked each time.
- That makes Layer 4 non-negotiable: no production change fires without explicit human approval.
- The Remediate agent's job ends at 'here is the proposed fix and why' — never at applying it.
- Every step is audited, so an approver sees the full chain: signal → diagnosis → proposed fix.
- This is exactly the 'human-agent trust exploitation' risk from S5 — the gate defends against a polished-but-wrong proposal.



Lab: Build the Self-Healing System

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define the four agents and the incident object flowing between them
- Pre-hook: validate and classify incoming signals
- Build Monitor → Diagnose → Remediate → Validate on your guarded stack
- Post-hook: route every proposed remediation through the Layer-4 human gate
- Attack: inject a plausible-but-harmful remediation; confirm the gate stops it
- Commit the system; run it through your S7 CI gate

SESSION DELIVERABLE

- A running self-healing system
- Full four-layer stack applied
-  Mandatory human gate before any change



End of S8 — Three Systems, Fully Guarded

✓ Checkpoint — you should now have

- A self-healing system running end-to-end
- A mandatory human gate before any production change
- The full four-layer stack applied to the hardest case
- Proof: a harmful remediation is caught at the gate
- The system passing your own CI security gate

💡 TRAINER TIP

This is the capstone-worthy system. If time is tight across the day, protect this session — it's where autonomy and governance meet, which is the whole thesis of the programme.



SESSION S9 · 75 MIN

Governance, Licensing & Data Protection

The non-code layer that makes everything shippable: data classification, licensing decisions, and the audit trail that proves compliance.



Governance, Licensing & Data Protection

Session objectives

- **Classify data (PUBLIC / INTERNAL / CONFIDENTIAL) and enforce it in code**
- Understand model licensing: open-source $\neq$ on-prem; hosting matters
- Use the audit trail to prove what an agent did and why
- Apply context-window hygiene: strip CONFIDENTIAL fields before inference

FORMAT

60% lab / 40% theory

DURATION

75 minutes

BUILDS ON

S2 audit + S6 L4

OUTPUT

Data-protection policy in code



Data Classification & Context Hygiene

Classify, then enforce

- Every input is **PUBLIC, INTERNAL, or CONFIDENTIAL**
- Classification happens at the Layer-1 input guard
- **CONFIDENTIAL** fields are stripped before they reach the model
- The context window is a data-leak surface — treat it as one
- Blind-paste prevention: no raw dump into a prompt

Licensing — the S9 decision tree

- Open-source model ≠ on-prem model — hosting is a **separate question**
- Apache 2.0 / MIT: safe for production use
- Restricted licenses: review before shipping
- Externally-hosted 'open' models still send data off-box
- Every choice is logged as a governance decision



The Audit Trail Is the Proof

Everything you built logs — S9 is where those logs become governance.

- Every tool call, every guardrail decision, every human approval is recorded (you built this in S2 and S6).
- The audit trail answers: what did the agent do, on what data, and who approved it?
- For a regulated environment, 'the agent decided' is not an answer — the trail is.
- Context-window hygiene + classification means CONFIDENTIAL data never silently enters a prompt.
- This is what turns a working demo into something a compliance team will actually approve.



Lab: Enforce Governance in Code

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define your data classification scheme and the fields each level covers
- Pre-hook: classify every input and tag it at the Layer-1 guard
- Build context-hygiene: strip CONFIDENTIAL fields before inference
- Post-hook: confirm no CONFIDENTIAL data appears in any output or log-in-clear
- Attack: submit CONFIDENTIAL data and trace it — confirm it's stripped and audited
- Commit the data-protection policy as enforced code

SESSION DELIVERABLE

- Data classification enforced in code
- Context-window hygiene working
-  Full audit trail of every decision



End of S9 — Shippable, Not Just Working

✓ Checkpoint — you should now have

- Data classification enforced at the input guard
- CONFIDENTIAL fields stripped before inference
- A licensing decision tree you can apply to any model
- An audit trail that proves what every agent did
- The governance policy committed as code

💡 TRAINER TIP

Frame this as the difference between 'it works' and 'legal will let us ship it.' For a Philips audience, this session is often the one that unlocks real deployment — give it the weight it deserves.

SESSION S10 · 120 MIN

Capstone Build & Showcase



You bring it all together: take one system to production-readiness with the full stack, then show it — including how it survives attack.



Capstone Build & Showcase

Session objectives

- Take one of your systems to production-ready with the full stack
- Apply all four guardrail layers, the CI gate, and governance
- Prove it survives the attack catalogue live
- Present it — architecture, guardrails, and a live attack that fails

FORMAT

90% build & present

DURATION

120 minutes

BUILDS ON

Everything

OUTPUT

A showcased, hardened system



The Capstone Brief

You choose the system; the bar is the same for everyone.

- **Pick one of your three systems and take it to production-ready.**
- **It must pass all four guardrail layers, your CI security gate, and the governance policy.**
- **It must survive the full S5 attack catalogue — demonstrated live.**
- **Your showcase covers: the architecture, where each guardrail sits, and one attack that visibly fails.**
- **The deliverable is not a slide — it's a running, hardened system plus its agent specification doc.**



Capstone: The Final Cycle

Every lab follows the same six-step cycle:



What you'll do in this lab

- Define production-ready for your chosen system — what 'done' means
- Pre-hook: confirm input classification and validation are complete
- Build the last mile — wire the full stack, pass your own CI gate
- Post-hook: run governance checks; confirm the audit trail is complete
- Attack & Validate: run the whole S5 catalogue live in your showcase
- Commit the final system + its agent specification document

SESSION DELIVERABLE

- A hardened, production-ready system
- The agent specification document
-  Live proof it survives every attack



What You're Taking Home

Your artefacts

- Two complete agent systems (documentation + content)
- A self-healing system with a governance gate
- A custom MCP server with auth + audit
- The four-layer guardrail stack as reusable code
- Six security scripts + a CI security gate
- An agent specification document format

Your new habits

- Build first, guardrail at every seam
- Never trust agent output blindly — validate it
- Automate the checks; a manual guardrail decays
- Classify data before it reaches a model
- Open-source model ≠ on-prem — hosting is a decision
- The audit trail is what makes it shippable

Thank You

You came agent-aware. You leave able to ship agents safely.
